

Isolating and Measuring Board Power with an INA226

1. What this rig measures, and what it does not

The shunt goes in series with the **DC input to the carrier board**, upstream of everything on it. The resulting number is **whole-device energy**: the SoC (GPU, CPU, and other accelerators together — there is no discrete GPU to separate), LPDDR5, the carrier board's own regulators, NVMe, and the fan.

Three exclusions to state explicitly in your methods, because a reviewer will look for them:

- **The AC/DC brick's losses are excluded.** You are measuring on the DC side, so the ~10–15% conversion loss in the 19 V adapter is upstream of your shunt and invisible. This is the correct boundary for your paper — it matches how the A100's NVML counter excludes PSU losses — but say so.
- **You cannot decompose this into a GPU share.** The number is the board. Any per-component attribution would have to come from the onboard INA3221 rails, which is precisely the software path you are validating against this one.
- **The INA226 and its microcontroller are outside the boundary** — by construction, see §2. Their consumption never appears in the measurement.

Carrier reference: the Orin Nano developer kit carrier (which supports all Orin NX modules) accepts 9–20 V on a 5.5 mm × 2.5 mm barrel jack, ships with a 19 V supply, and is rated to a maximum of 3.5 A. USB-C on this carrier is recovery-mode only and is not a power input. All sizing below assumes the 19 V barrel input.

2. Three things that must be isolated

"Isolated microcontroller" in your briefing is doing more work than it looks. There are three distinct isolation requirements, and only the first is obvious.

2.1 The sampler must not draw power from the measured domain

You cannot sample the Jetson's power from the Jetson. A sampling loop running on the Orin's own CPU consumes energy that flows through your shunt, so the act of measuring would inflate the measurement — and inflate it *more* during idle windows than during inference, since the sampler's relative contribution is largest when the device is otherwise quiet. That would corrupt the idle baseline you subtract from every call. A separate MCU is mandatory, not a convenience.

The subtle version of the same mistake: do not plug the sampling MCU into one of the Jetson's USB ports. USB VBUS is sourced by the Jetson, downstream of your shunt — so the MCU's own consumption (plus the INA226's) would flow through the shunt and be attributed to inference. **Power the MCU from the logging laptop**, and connect only the trigger line and ground to the Jetson.

The INA226 itself is powered from the MCU's 3.3 V rail. Its ~330 μ A quiescent draw therefore sits entirely outside the measured domain. Its `IN+` / `IN-` pins see a 19 V common-mode voltage, comfortably inside the part's 0–36 V common-mode range, which is independent of its supply voltage. This is exactly the topology the part is designed for.

2.2 High-side sensing is mandatory here, not merely preferable

A low-side shunt (in the return path) is electrically simpler, but it lifts the Jetson's ground above the supply ground by $I \times R$ — about 30 mV at 1.5 A with the shunt sized below. That offset is small, but it **varies with load**.

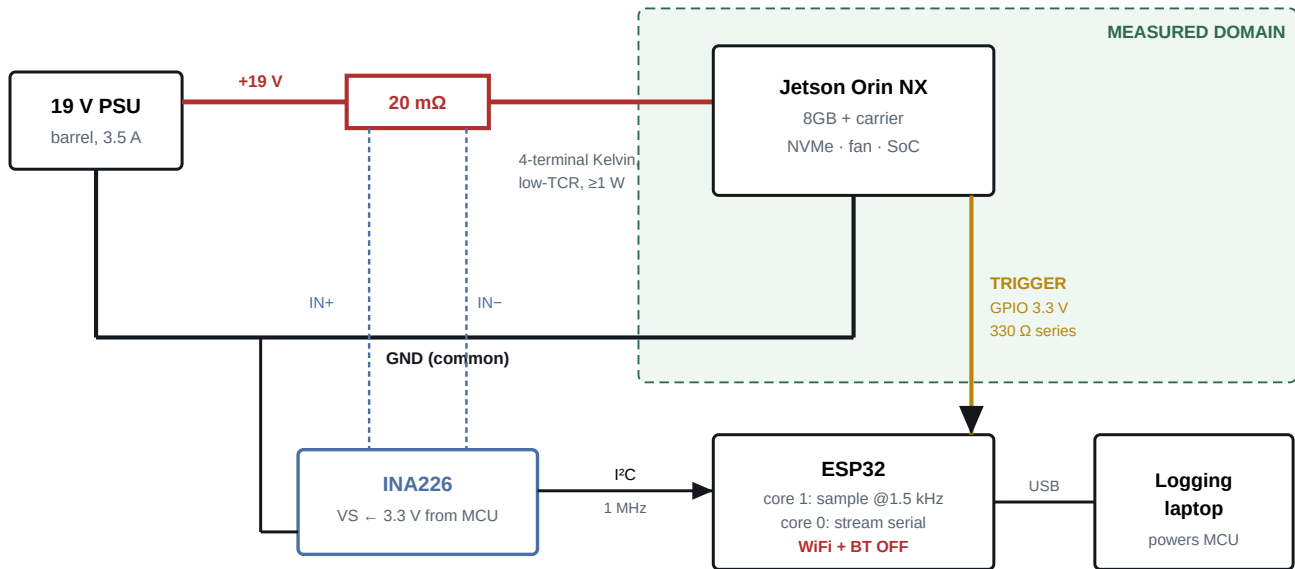
You are running a GPIO logic line from the Jetson to the sampling MCU. A ground reference that moves with load moves the logic threshold of that trigger line — and it moves *in step with the very quantity you are measuring*. Low-side sensing would couple your load current into your call-boundary signal. **Put the shunt in the +19 V line.**

2.3 Grounding, and the one loop worth breaking

The INA226 must be referenced to the same ground as the supply negative, and it is powered from the MCU, so the MCU and Jetson necessarily share a ground node. This is fine: the only current in that wire is the INA226's supply return and a few microamps of trigger-line drive. Do not attempt galvanic isolation of the trigger line alone — it would accomplish nothing, since the ground is already common through the sensor.

The loop actually worth breaking is the mains one: the laptop is earthed, the Jetson's PSU is earthed, and your ground wire closes the path between them. Either **run the laptop on battery during measurement blocks**, or insert a USB isolator (ADuM3160-class) between laptop and MCU. Cheap insurance against a noise source that would be very hard to diagnose after the fact.

3. Electrical topology



Shaded region = everything the shunt sees. The INA226, the MCU, and the laptop sit outside it and contribute nothing to the measurement. Sense leads (dashed) carry no current — they are the Kelvin connection.

Note the MCU is powered by the laptop, NOT by a Jetson USB port.

High-side shunt on the 19 V input. Ground is common; only the INA226's supply return and trigger drive flow in it.

4. Sizing the shunt and the calibration register

Expected draw: the Orin NX 8GB at the 20 W `nvpmodel` mode, plus carrier, NVMe, and a fan pinned at 100%, should land around 25–30 W at the DC input — roughly **1.3–1.6 A at 19 V**. Size full-scale to the carrier's 3.5 A rating so transients and inrush can never clip.

Quantity	Value	Derivation
Shunt full-scale voltage	±81.92 mV	INA226 fixed
Shunt LSB	2.5 μV	INA226 fixed (16-bit)
R_{SHUNT}	20 mΩ	81.92 mV ÷ 4.096 A
Full-scale current	4.096 A	covers the 3.5 A carrier rating
Current_LSB	125 μA	4.096 A ÷ 2 ¹⁵
CAL register	2048	0.00512 ÷ (125 μA × 20 mΩ)
Power_LSB	3.125 mW	25 × Current_LSB (fixed ratio)
Effective power resolution	≈ 2.4 mW	125 μA × 19 V
Shunt drop at 1.5 A	30 mV	0.16% of 19 V — does not perturb the supply
Shunt dissipation at 1.5 A	45 mW	245 mW at the 3.5 A limit → use a ≥1 W part, 2 W preferred

Alternative: 25 mΩ gives 3.277 A full scale and a 100 μA Current_LSB — slightly better resolution, less headroom. CAL is 2048 either way, since only the product Current_LSB × R_{SHUNT} enters the equation. Take 20 mΩ unless you have a specific reason not to.

Check what your breakout actually has on it. The common INA226 modules (CJMCU-226 and its clones) ship with a **100 mΩ** shunt. That gives a full-scale range of 0.819 A — your Jetson would **saturate it at roughly half load**, and the register would simply pin. It would also drop 160 mV and dissipate a quarter watt right next to your sense amplifier. Either replace the resistor or buy a board designed for an external shunt.

4.1 Kelvin connection — the error most likely to bite you

At 20 mΩ, the resistances you are ignoring are not negligible. A few millimetres of PCB trace, a solder fillet, and a crimp can easily add 2 mΩ, which is a **10% gain error** — and it is invisible, because the reading looks perfectly plausible.

- Use a **4-terminal (Kelvin) current-sense resistor**: two heavy terminals for the current path, two separate sense terminals.
- If you only have a 2-terminal part, solder the sense wires *directly to the resistor's end caps*, inboard of the current-carrying joints — never to the wire or pad beyond them.
- The sense pair carries essentially no current, so use thin wire and **twist it**; keep the INA226 physically close to the shunt.
- Current path: 18 AWG or heavier, kept short.

Temperature coefficient. Your run is six days continuous and your calibration may be a week older than the data. Specify a low-TCR current-sense part (≤50 ppm/°C; 15–25 ppm/°C available in Vishay WSL/WSLP, Bourns CSS, Susumu KRL families). Log ambient temperature alongside everything else so drift can be checked rather than assumed.

5. Configuring for 1.5 kHz

Your briefing specifies roughly 1–1.5 kHz. The INA226 hits the top of that range cleanly:

$$t_{\text{cycle}} = (\text{shunt CT} + \text{bus CT}) \times \text{averaging} = (332 \mu\text{s} + 332 \mu\text{s}) \times 1 = 664 \mu\text{s} \rightarrow 1506 \text{ Hz}$$

Register	Setting	Reason
Config (0x00) — MODE	Shunt + bus, continuous	Free-running; no per-sample trigger overhead
Config — VSHCT	332 μs	Hits 1.5 kHz with bus enabled
Config — VBUSCT	332 μs	As above
Config — AVG	1	Averaging in the MCU instead, so no information is discarded before you see it
Calibration (0x05)	2048	Per §4
Mask/Enable (0x06)	Conversion Ready enabled	Drives ALERT — see below

5.1 Use the ALERT pin, do not free-run poll

The Conversion Ready flag asserts once conversion, averaging, and multiplication are complete, and can be routed to the ALERT pin. Wire that to an MCU interrupt.

If you instead poll on a software timer, you will drift relative to the INA226's internal conversion clock and will intermittently read the same sample twice or skip one entirely. Either error corrupts an energy integral silently — duplicated samples inflate it, skipped samples deflate it, and neither leaves a trace in the data. **Sample exactly when the part says a new value exists.**

5.2 I²C bandwidth

Two register reads per cycle (shunt 0x01, bus 0x02) is roughly 100 bits of bus traffic including addressing and overhead. At 400 kHz that is ~250 μ s of a 664 μ s budget — workable but with little margin for interrupt latency.

Run the bus at 1 MHz (the part supports high-speed mode to 2.94 MHz), which drops it to ~100 μ s.

Optimisation worth taking: the 19 V rail is regulated and slow-moving, while current is the fast-varying quantity. Read **shunt every cycle and bus voltage only every ~100th cycle** (≈ 15 Hz), then interpolate bus voltage in post-processing. This halves bus traffic and costs well under 0.1% accuracy. You can then also drop bus CT and push the shunt-only rate higher if you want headroom.

Why a delta-sigma part matters here. The INA226's ADC *integrates over the whole conversion window* rather than taking an instantaneous snapshot. The Jetson's DC-DC converters switch in the hundreds of kHz; a sample-and-hold ADC running at 1.5 kHz would alias that switching ripple straight into your data as a plausible-looking low-frequency artifact. The integrating converter averages it out within each 332 μ s window instead. This is a real methodological point and worth one sentence in the paper — it is the reason 1.5 kHz is adequate rather than dangerously undersampled.

6. The trigger line and time alignment

This is what makes an energy number belong to *one call*. The principle in your briefing is right: the trigger and the current must be read by the same device, off the same clock, so they cannot drift apart.

6.1 Electrical

- Jetson 40-pin header GPIO, 3.3 V logic → MCU input. Both are 3.3 V, so no level shifting. Add a **330 Ω series resistor** to survive a wiring mistake.
- Drive it with **libgpiod** (character device), not the deprecated sysfs interface — lower and far more consistent latency.
- Consider a second pin as a *block marker* (high for the duration of an idle-measurement window, or to mark topology boundaries). Pins are free; re-running six days of data because you cannot segment it is not.

6.2 Capture both edges precisely

Rather than only recording trigger state alongside each ADC sample, timestamp the trigger's rising and falling **edges** with a GPIO interrupt on the same hardware timer that timestamps samples. Edge resolution then drops from one sample period (664 μs) to ~1 μs, and you can interpolate the partial samples at the call boundaries.

The gain is small — 664 μs of edge uncertainty on a 15 s call is 0.004% — but it is free, and it lets you state a precise attribution error rather than a bound.

6.3 The latency you actually have to report

Between the orchestrator setting the pin high and the GPU beginning real work, several things happen: the `gpiod` call traverses the kernel, the HTTP POST is issued and parsed, and llama.cpp tokenizes and schedules. So the trigger window **brackets slightly more than the model call**.

This is not a problem, and here is the argument. The dispatch overhead is *identical across all four conditions and all three temperatures* — it is a property of your orchestrator, not of the topology. It therefore cannot bias any between-condition comparison, which is the only thing your paper claims. It slightly inflates absolute per-call energy in a uniform way. State the measured magnitude, note that it is constant, and move on.

Characterise it (calibration task, §11 of the design doc): have the orchestrator toggle the pin 10,000 times in a loop with a known intended pulse width, and have the MCU measure what actually arrived. Report the P50 and P99 of the discrepancy. Then reduce the tail by pinning the orchestrator to an isolated core (`taskset`) and giving it `SCHED_FIFO` priority — but apply that configuration identically to every condition, since core isolation itself changes the device's power profile.

7. Sampler firmware architecture

7.1 ESP32 — the conditions that make it fine

The standard objection to an ESP32 as a timing reference is real but *specific*: the WiFi and Bluetooth stacks run high-priority tasks and take interrupts you do not control, producing stalls of hundreds of microseconds to milliseconds. At a 664 μs sample period that is fatal — you would miss conversion-ready events and never know.

That objection disappears entirely if the radios are off. With WiFi and BT disabled, an ESP32 is a dual-core 240 MHz microcontroller with a 64-bit microsecond timer and a hardware I²C peripheral that reaches 1 MHz — comfortably enough for this job. **Use the ESP32; just build it with the radios disabled and observe the six rules below.**

Rule	Why
WiFi and BT never initialised	The entire basis of the timing argument. Do not "just leave it on for convenience" — stream over the wire instead. This is non-negotiable.
Pin sampling to core 1, max FreeRTOS priority; serial transport on core 0	Separates deterministic acquisition from blocking I/O, the same split the design needs on any dual-core part.
All ISRs marked <code>IRAM_ATTR</code>	Otherwise the handler lives in flash and a cache miss adds unbounded latency — or faults outright during a flash operation. This is the most common ESP32 timing bug.
No flash writes during sampling (no NVS, no SPIFFS logging)	Flash operations disable the instruction cache and stall both cores. Stream data out; never buffer it to onboard flash.
Serial at 921600 baud or higher , or native USB-CDC on ESP32-S3	You need ~12 kB/s sustained. 115200 baud is ~11.5 kB/s raw — under framing overhead it will not keep up, and the failure mode is silent buffer overflow.
Feed or disable the task watchdog on the sampling core	A tight max-priority loop will otherwise trip the watchdog and reset the device mid-block.

I²C specifics. Use the ESP-IDF 5.x `driver/i2c_master.h` API rather than the legacy `driver/i2c.h`, which has a long history of edge-case bugs. Running at 1 MHz, size the pull-ups at **1 k Ω –2.2 k Ω** , not the usual 4.7 k Ω — rise time is the limiting factor at that speed. Keep the bus under ~20 cm and keep SDA/SCL away from the 19 V current path.

Timebase. `esp_timer_get_time()` gives 64-bit microseconds since boot from a hardware timer. That is your single clock — use it for ADC samples and trigger edges alike, and never mix in any other time source.

Variant note. The original ESP32 has no native USB and relies on a CP2102/CH340 UART bridge, so 921600 baud is your ceiling-ish. An **ESP32-S3** has native USB-CDC and is the easier choice if you have one; the S3 also has a slightly cleaner interrupt story. Either works.

7.2 Architecture

```
core 1 - acquisition task, max priority, never blocks
  on ALERT (conversion ready) GPIO interrupt [IRAM_ATTR]
    t = esp_timer_get_time()           // single timebase, microseconds
    sh = i2c_read16(0x01)              // shunt voltage register
    if (n % 100 == 0) bus = i2c_read16(0x02)
    ring_push({t, sh, bus_latest, trigger_state, seq++})

  on trigger GPIO edge interrupt [IRAM_ATTR]
    ring_push_event({esp_timer_get_time(), edge_direction, seq++})

core 0 - transport task, may block freely
  drain ring buffer -> UART/USB-CDC -> laptop
  binary framing, sequence number on every record
```

- **One timebase only.** Every timestamp — samples and trigger edges alike — comes from the same free-running microsecond counter. This is the entire basis of your attribution claim.
- **Sequence-number every record** and check for gaps on the laptop side. A dropped sample must be detectable, not silently absorbed into an integral.
- **Data rate:** $\sim 1.5 \text{ kHz} \times 8 \text{ bytes} \approx 12 \text{ kB/s}$ sustained. See the baud-rate rule in §7.1.
- **Never do floating-point or unit conversion on the MCU.** Log raw registers and convert offline. Raw values are re-processable when you later revise a calibration constant; converted values are not.
- **Power the ESP32 from the laptop**, per §2.1 — that rule applies whatever the MCU.

8. Computing energy per call

Integrate instantaneous power over the trigger window, trapezoidal rule:

$$E_{\text{call}} = \sum_k \frac{1}{2} (P_k + P_{k+1}) \cdot \Delta t_k \quad \text{where} \quad P_k = V_{\text{bus}}(t_k) \cdot I_k$$

with k ranging over samples between the rising and falling trigger edges, V_{bus} linearly interpolated from its slower sample series, and the partial samples at each boundary scaled by the fraction of the interval inside the window.

Then subtract the idle baseline measured with the server loaded and resident (see §2.2 of the design doc — subtracting a server-down baseline is a systematic error that inflates every call by a constant):

$$E_{\text{net}} = E_{\text{call}} - \bar{P}_{\text{idle}} \cdot (t_{\text{fall}} - t_{\text{rise}})$$

Report both. E_{call} is what the device actually consumed; E_{net} is what the call added. The proxy regressions should be run on both, because they can give different answers — and if they do, that difference is worth a paragraph, since papers in this space are rarely explicit about which one they mean by "energy."

9. Calibration against a reference meter

Your briefing commits to a rig "calibrated against a reference meter rather than trusted as-is." That commitment is what separates this from the onboard counters you are validating, so it needs to be a documented procedure with a reported residual.

1. **Zero/offset.** With the load disconnected, record the shunt register over ~ 60 s. The mean is your offset; subtract it everywhere. The standard deviation is your noise floor — report it.
2. **Multi-point gain.** Using an electronic load or a set of power resistors on the 19 V supply, establish steady currents at roughly 0.2, 0.5, 1.0, 1.5, 2.0, 2.5 A. At each point, record the INA226 mean over 30 s alongside a calibrated bench DMM in series. Regress INA226 against DMM; the slope is your gain correction, the intercept confirms the offset.
3. **Report the residual.** Give the R^2 , the maximum residual across points, and the resulting $\pm\%$ accuracy claim. This number is what a reviewer will quote.
4. **Bus voltage.** Calibrate separately against the DMM in voltmeter mode. It is a small correction but it multiplies directly into every power value.
5. **Repeat at the end of the campaign.** Re-run the sweep after the six weeks and report the drift between the two calibrations. This bounds the systematic error over the run and is the single most convincing thing you can put in a measurement methods section.

9.1 Independent sanity checks

Check	Expectation
Known resistive load	$P = V^2/R$ within the calibrated tolerance. A 100 Ω 10 W resistor at 19 V $\rightarrow$ 3.61 W, 190 mA.
Long-window cross-check	Integrated energy over 10 minutes against a commercial inline DC meter — agreement well inside your stated tolerance.
Whole exceeds the parts	INA226 whole-device power must always exceed the sum of the onboard INA3221 rails. If it ever does not, something is wrong with the shunt path.
Idle stability	Consecutive idle windows within a block should agree closely. Growing divergence means thermal drift is not being controlled by the cooldown gate.
Trigger sanity	$\Sigma(\text{trigger-high durations}) + \Sigma(\text{gaps}) = \text{block wall clock}$, with no overlapping high periods — which also verifies the serialisation guarantee.

10. Error budget

Build this table with your own measured numbers and put it in the paper. It is the artifact that makes the whole-device measurement credible.

Source	Typical magnitude	Handling
Shunt tolerance	± 0.5 to $\pm 1\%$	Systematic gain — removed by the §9 calibration
Shunt TCR	25 ppm/°C $\times \Delta T$	Low-TCR part; log ambient; bracket with start/end calibration
Parasitic path resistance	up to 10% if botched	Kelvin sensing — the dominant risk, and entirely avoidable
INA226 gain error	$\pm 0.1\%$ typ	Absorbed by calibration
INA226 offset	$\approx \pm 10 \mu\text{V}$ shunt	Measured at zero load and subtracted
ADC quantization	125 μA ($\approx 2.4 \text{ mW}$)	Negligible against a $\sim 25 \text{ W}$ load
Bus voltage interpolation	$< 0.1\%$	Rail is regulated and slow; 15 Hz is ample
Trigger edge quantization	$\pm 1 \mu\text{s}$ with edge capture	0.00001% of a 15 s call
Dispatch overhead in window	~ 0.1 – 2 ms , tail outliers	Constant across conditions — no between-condition bias; report P50/P99
Idle-baseline uncertainty	set by idle stability	Interleaved idle windows; model drift rather than assuming it flat

11. Bill of materials

Item	Specification	Notes
Current sense amplifier	INA226	Breakout with <i>external</i> shunt provision, or bare IC
Shunt resistor	20 m Ω , $\pm 0.5\%$, $\leq 50 \text{ ppm}/^\circ\text{C}$, $\geq 1 \text{ W}$, 4-terminal	Vishay WSLP / Bourns CSS / Susumu KRL. Do not use the 100 mΩ part fitted to most modules.
Microcontroller	ESP32 (ESP32-S3 preferred)	Radios disabled in firmware — see §7.1. S3 adds native USB-CDC.
I ² C pull-ups	$2 \times 1 \text{ k}\Omega$ – $2.2 \text{ k}\Omega$	Not 4.7 k Ω — rise time limits at 1 MHz
Barrel jack pigtails	$5.5 \times 2.5 \text{ mm}$, male + female	Cut the positive lead to insert the shunt inline
Wire	18 AWG current path; thin twisted pair for sense	Keep the current path short
USB isolator	ADuM3160-class	Optional — or simply run the laptop on battery
Reference DMM	Calibrated bench meter, 5½ digit or better	Borrow; needed only for the calibration sessions

If you have not bought the sensor yet, consider the INA228. It is the same architecture with 20-bit resolution and — directly relevant here — a hardware **energy accumulator register** that integrates in silicon. That removes numerical integration from your pipeline entirely and eliminates the dropped-sample failure mode, which is attractive for a paper whose contribution *is* the energy measurement. If the INA226 is already in hand, it is entirely defensible; correct integration as described in §8 gets you there. Do not churn hardware over this.

12. Methods checklist

Everything below should appear in the paper. Reviewers of energy-measurement work look for exactly these, and their absence reads as an omission rather than a choice.

- Measurement boundary: DC input to carrier board; AC/DC adapter losses excluded; no GPU-only decomposition possible.
- Sensor part number, shunt value, tolerance, and temperature coefficient.
- Sampling rate and the conversion-time / averaging configuration that produces it.
- That the ADC integrates over each conversion window rather than sampling instantaneously (i.e. switching ripple is averaged, not aliased).
- Calibration procedure, number of points, residual error, and the start-to-end drift.
- Zero-load offset and noise floor.
- Call-attribution mechanism, shared timebase, and the measured dispatch-overhead distribution — with the note that it is constant across conditions.
- That the sampler draws no power from the measured domain, and that its radios were disabled.
- Idle-baseline definition (server loaded and resident) and how it was interleaved.
- Whether reported energy is `E_call` or `E_net` — ideally both.

Sources: TI INA226 datasheet (SBOS547) · NVIDIA Jetson Orin Nano Developer Kit Carrier Board Specification SP-11324-001 · NVIDIA Jetson Linux Developer Guide, Platform Power and Performance.